# lab_gene_partial

October 8, 2025

## 1 Lab: Logistic Regression for Gene Expression Data

In this lab, we use logistic regression to predict biological characteristics ("phenotypes") from gene expression data. In addition to the concepts in breast cancer demo, you will learn to: * Handle missing data * Perform multi-class logistic classification * Create a confusion matrix * Use L1-regularization for improved estimation in the case of sparse weights (Grad students only)

### 1.1 Background

Genes are the basic unit in the DNA and encode blueprints for proteins. When proteins are synthesized from a gene, the gene is said to "express". Micro-arrays are devices that measure the expression levels of large numbers of genes in parallel. By finding correlations between expression levels and phenotypes, scientists can identify possible genetic markers for biological characteristics.

The data in this lab comes from:

https://archive.ics.uci.edu/ml/datasets/Mice+Protein+Expression

In this data, mice were characterized by three properties: * Whether they had down's syndrome (trisomy) or not * Whether they were stimulated to learn or not * Whether they had a drug memantine or a saline control solution.

With these three choices, there are 8 possible classes for each mouse. For each mouse, the expression levels were measured across 77 genes. We will see if the characteristics can be predicted from the gene expression levels. This classification could reveal which genes are potentially involved in Down's syndrome and if drugs and learning have any noticeable effects.

### 1.2 Load the Data

We begin by loading the standard modules.

[36]:
```
import pandas as pd
import numpy as np
import matplotlib
import matplotlib.pyplot as plt
%matplotlib inline
from sklearn import linear_model, preprocessing
from sklearn.linear_model import LogisticRegression
```

Use the `pd.read_excel` command to read the data from

https://archive.ics.uci.edu/ml/machine-learning-databases/00342/Data_Cortex_Nuclear.xls

into a dataframe `df`. Use the `index_col` option to specify that column 0 is the index. Use the `df.head()` to print the first few rows.

[37]:
```
# TODO
# read data from https://archive.ics.uci.edu/ml/machine-learning-databases/
 ↪00342/Data_Cortex_Nuclear.xls
df = pd.read_excel(
    'https://archive.ics.uci.edu/ml/machine-learning-databases/00342/
 ↪Data_Cortex_Nuclear.xls',
    index_col=0
)

# print first few rows
df.head()
```

[37]:

| MouseID | DYRK1A_N | ITSN1_N | BDNF_N | NR1_N | NR2A_N | pAKT_N | pBRAF_N | \ |
|---|---|---|---|---|---|---|---|---|
| 309_1 | 0.503644 | 0.747193 | 0.430175 | 2.816329 | 5.990152 | 0.218830 | 0.177565 | |
| 309_2 | 0.514617 | 0.689064 | 0.411770 | 2.789514 | 5.685038 | 0.211636 | 0.172817 | |
| 309_3 | 0.509183 | 0.730247 | 0.418309 | 2.687201 | 5.622059 | 0.209011 | 0.175722 | |
| 309_4 | 0.442107 | 0.617076 | 0.358626 | 2.466947 | 4.979503 | 0.222886 | 0.176463 | |
| 309_5 | 0.434940 | 0.617430 | 0.358802 | 2.365785 | 4.718679 | 0.213106 | 0.173627 | |

| MouseID | pCAMKII_N | pCREB_N | pELK_N | … | pCFOS_N | SYP_N | H3AcK18_N | \ |
|---|---|---|---|---|---|---|---|---|
| | | | | … | | | | |
| 309_1 | 2.373744 | 0.232224 | 1.750936 | … | 0.108336 | 0.427099 | 0.114783 | |
| 309_2 | 2.292150 | 0.226972 | 1.596377 | … | 0.104315 | 0.441581 | 0.111974 | |
| 309_3 | 2.283337 | 0.230247 | 1.561316 | … | 0.106219 | 0.435777 | 0.111883 | |
| 309_4 | 2.152301 | 0.207004 | 1.595086 | … | 0.111262 | 0.391691 | 0.130405 | |
| 309_5 | 2.134014 | 0.192158 | 1.504230 | … | 0.110694 | 0.434154 | 0.118481 | |

| MouseID | EGR1_N | H3MeK4_N | CaNA_N | Genotype | Treatment | Behavior | class |
|---|---|---|---|---|---|---|---|
| 309_1 | 0.131790 | 0.128186 | 1.675652 | Control | Memantine | C/S | c-CS-m |
| 309_2 | 0.135103 | 0.131119 | 1.743610 | Control | Memantine | C/S | c-CS-m |
| 309_3 | 0.133362 | 0.127431 | 1.926427 | Control | Memantine | C/S | c-CS-m |
| 309_4 | 0.147444 | 0.146901 | 1.700563 | Control | Memantine | C/S | c-CS-m |
| 309_5 | 0.140314 | 0.148380 | 1.839730 | Control | Memantine | C/S | c-CS-m |

[5 rows x 81 columns]

This data has missing values. The site:

http://pandas.pydata.org/pandas-docs/stable/missing_data.html

has an excellent summary of methods to deal with missing values. Following the techniques there, create a new data frame `df1` where the missing values in each column are filled with the mean values from the non-missing values.

[38]:
```
# TODO
# Fill missing values with mean of each column
# First convert non-numeric columns to numeric
numeric_cols = df.select_dtypes(include=[np.number]).columns
df1 = df.copy()
df1[numeric_cols] = df1[numeric_cols].fillna(df1[numeric_cols].mean())

df1.head()
```

[38]:

| MouseID | DYRK1A_N | ITSN1_N | BDNF_N | NR1_N | NR2A_N | pAKT_N | pBRAF_N | \ |
|---|---|---|---|---|---|---|---|---|
| 309_1 | 0.503644 | 0.747193 | 0.430175 | 2.816329 | 5.990152 | 0.218830 | 0.177565 | |
| 309_2 | 0.514617 | 0.689064 | 0.411770 | 2.789514 | 5.685038 | 0.211636 | 0.172817 | |
| 309_3 | 0.509183 | 0.730247 | 0.418309 | 2.687201 | 5.622059 | 0.209011 | 0.175722 | |
| 309_4 | 0.442107 | 0.617076 | 0.358626 | 2.466947 | 4.979503 | 0.222886 | 0.176463 | |
| 309_5 | 0.434940 | 0.617430 | 0.358802 | 2.365785 | 4.718679 | 0.213106 | 0.173627 | |

| MouseID | pCAMKII_N | pCREB_N | pELK_N | … | pCFOS_N | SYP_N | H3AcK18_N | \ |
|---|---|---|---|---|---|---|---|---|
| | | | | … | | | | |
| 309_1 | 2.373744 | 0.232224 | 1.750936 | … | 0.108336 | 0.427099 | 0.114783 | |
| 309_2 | 2.292150 | 0.226972 | 1.596377 | … | 0.104315 | 0.441581 | 0.111974 | |
| 309_3 | 2.283337 | 0.230247 | 1.561316 | … | 0.106219 | 0.435777 | 0.111883 | |
| 309_4 | 2.152301 | 0.207004 | 1.595086 | … | 0.111262 | 0.391691 | 0.130405 | |
| 309_5 | 2.134014 | 0.192158 | 1.504230 | … | 0.110694 | 0.434154 | 0.118481 | |

| MouseID | EGR1_N | H3MeK4_N | CaNA_N | Genotype | Treatment | Behavior | class |
|---|---|---|---|---|---|---|---|
| 309_1 | 0.131790 | 0.128186 | 1.675652 | Control | Memantine | C/S | c-CS-m |
| 309_2 | 0.135103 | 0.131119 | 1.743610 | Control | Memantine | C/S | c-CS-m |
| 309_3 | 0.133362 | 0.127431 | 1.926427 | Control | Memantine | C/S | c-CS-m |
| 309_4 | 0.147444 | 0.146901 | 1.700563 | Control | Memantine | C/S | c-CS-m |
| 309_5 | 0.140314 | 0.148380 | 1.839730 | Control | Memantine | C/S | c-CS-m |

[5 rows x 81 columns]

### 1.3 Binary Classification for Down's Syndrome

We will first predict the binary class label in `df1['Genotype']` which indicates if the mouse has Down's syndrome or not. Get the string values in `df1['Genotype'].values` and convert this to a numeric vector `y` with 0 or 1. You may wish to use the `np.unique` command with the `return_inverse=True` option.

[39]:
```
# TODO
y = np.unique(df1['Genotype'].values, return_inverse=True)[1]
```

As predictors, get all but the last four columns of the dataframes. Store the data matrix into `X` and the names of the columns in `xnames`.

[40]:
```
# TODO
xnames = df1.columns[:-4]
X = df1.iloc[:, :-4]
```

Split the data into training and test with 30% allocated for test. You can use the train

[41]:
```
from sklearn.model_selection import train_test_split

# TODO:
Xtr, Xts, ytr, yts = train_test_split(X, y, test_size=0.3, random_state=100)
```

Scale the data with the `StandardScaler`. Store the scaled values in `Xtr1` and `Xts1`.

[42]:
```
from sklearn.preprocessing import StandardScaler

# TODO
Xtr1 = StandardScaler().fit_transform(Xtr)
Xts1 = StandardScaler().fit_transform(Xts)
```

Create a `LogisticRegression` object `logreg` and `fit` on the scaled training data. Set the regularization level to `C=1e5` and use the optimizer `solver=liblinear`.

[43]:
```
# TODO
from sklearn.linear_model import LogisticRegression

logreg = LogisticRegression(C=1e5, solver='liblinear')
logreg.fit(Xtr1, ytr)
```

[43]:
```
LogisticRegression(C=100000.0, solver='liblinear')
```

Measure the accuracy of the classifer on test data. You should get around 94%.

[34]:
```
# TODO
yhat = logreg.predict(Xts1)
accuracy = np.mean(yhat == yts)
print(f"Accuracy: {accuracy:.2%}")
```

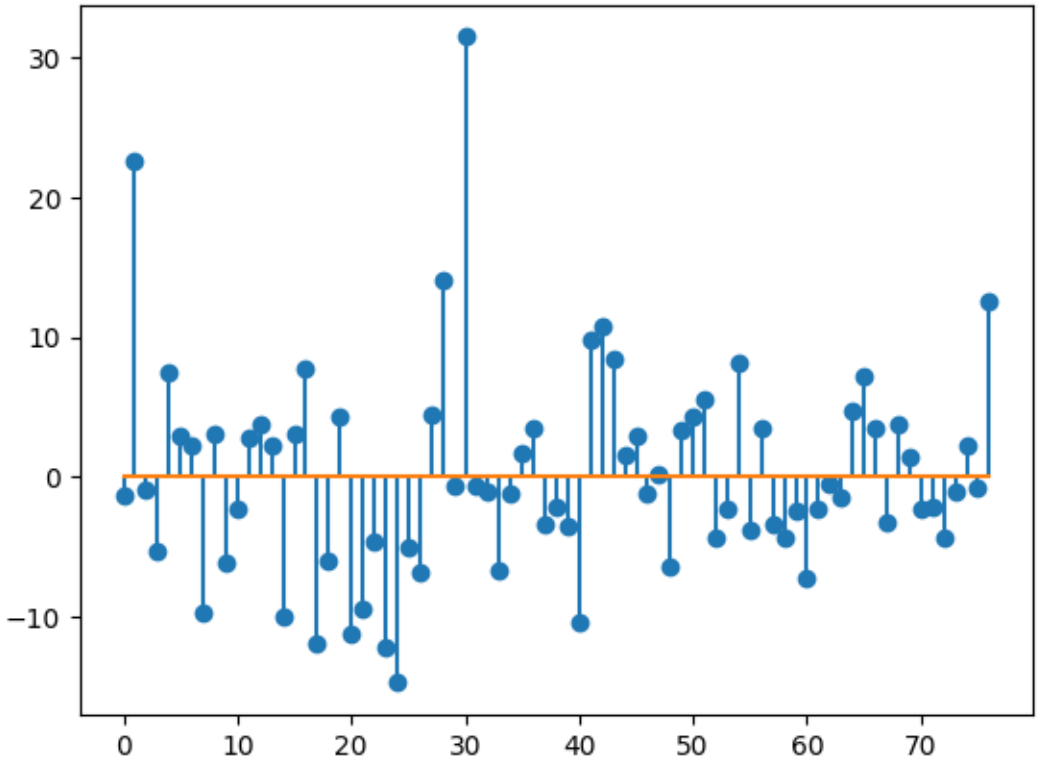
```
Accuracy: 96.30%
```

### 1.4 Interpreting the weight vector

Create a stem plot of the coefficients, `W` in the logistic regression model. Jse the `plt.stem()` function with the `use_line_collection=True` option. You can get the coefficients from `logreg.coef_`, but you will need to reshape this to a 1D array.

[49]:
```
# TODO
W = logreg.coef_.reshape(-1)
# `use_line_collection=True not supported in this version of matplotlib and␣
 ↪causing errors
plt.stem(W, linefmt='-', markerfmt='o', basefmt='-')
```

[49]: <StemContainer object of 3 artists>




You should see that `W[i]` is very large for a few components `i`. These are the genes that are likely to be most involved in Down's Syndrome. Below we will use L1 regression to enforce sparsity. Find the names of the genes for two components `i` where the magnitude of `W[i]` is largest.

[52]:
```
# TODO
# find the two components of i when W[i] is largest and second largest
largest_indices = np.argsort(W)[-2:]
largest_genes = xnames[largest_indices]
print(largest_genes[0])
print(largest_genes[1])
```

```
ITSN1_N
APP_N
```

### 1.5 Cross Validation

To obtain a slightly more accurate result, now perform 10-fold cross validation and measure the average precision, recall and f1-score. Note, that in performing the cross-validation, you will want to randomly permute the test and training sets using the **`shuffle`** option. In this data set, all the samples from each class are bunched together, so shuffling is essential. Print the mean precision, recall and f1-score and error rate across all the folds.

[57]:
```
from sklearn.model_selection import KFold
from sklearn.metrics import precision_recall_fscore_support
nfold = 10
kf = KFold(n_splits=nfold,shuffle=True)

# TODO
# ## Cross Validation

# Initialize arrays to store metrics
precision = np.zeros(nfold)
recall = np.zeros(nfold)
f1 = np.zeros(nfold)
error_rate = np.zeros(nfold)

# Perform k-fold cross validation
for i, (train_idx, test_idx) in enumerate(kf.split(X)):
    # Split data into train and test sets
    X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
    y_train, y_test = y[train_idx], y[test_idx]

    # Fit model on training data
    logreg = LogisticRegression(C=1e5, solver='liblinear')
    logreg.fit(X_train, y_train)

    # Make predictions on test data
    y_pred = logreg.predict(X_test)

    # Calculate metrics
    prec, rec, f1_score, _ = precision_recall_fscore_support(y_test, y_pred,␣
 ↪average='binary')
    precision[i] = prec
    recall[i] = rec
    f1[i] = f1_score
    error_rate[i] = 1 - np.mean(y_pred == y_test)

# Print mean metrics
print(f"Mean precision: {np.mean(precision):.3f} +/- {np.std(precision):.3f}")
print(f"Mean recall: {np.mean(recall):.3f} +/- {np.std(recall):.3f}")
print(f"Mean F1-score: {np.mean(f1):.3f} +/- {np.std(f1):.3f}")
print(f"Mean error rate: {np.mean(error_rate):.3f} +/- {np.std(error_rate):.
 ↪3f}")
```

```
Mean precision: 0.954 +/- 0.033
Mean recall: 0.964 +/- 0.032
Mean F1-score: 0.958 +/- 0.028
Mean error rate: 0.040 +/- 0.028
```

### 1.6 Multi-Class Classification

Now use the response variable in `df1['class']`. This has 8 possible classes. Use the `np.unique` funtion as before to convert this to a vector `y` with values 0 to 7.

[66]:
```
# TODO
y = np.unique(df1['class'].values, return_inverse=True)[1]
```

Fit a multi-class logistic model by creating a `LogisticRegression` object, `logreg` and then calling the `logreg.fit` method.

[67]:
```
# TODO
logreg = LogisticRegression(C=1e5, solver='liblinear')
logreg.fit(Xtr1, ytr)
```

[67]:
```
LogisticRegression(C=100000.0, solver='liblinear')
```

Now perform 10-fold cross validation, and measure the confusion matrix `C` on the test data in each fold. You can use the `confustion_matrix` method in the `sklearn` package. Add the confusion matrix counts across all folds and then normalize the rows of the confusion matrix so that they sum to one. Thus, each element `C[i,j]` will represent the fraction of samples where `yhat==j` given `ytrue==i`. Print the confusion matrix. You can use the command

`print(np.array_str(C, precision=4, suppress_small=True))`

to create a nicely formatted print. Also print the overall mean and SE of the test accuracy across the folds.

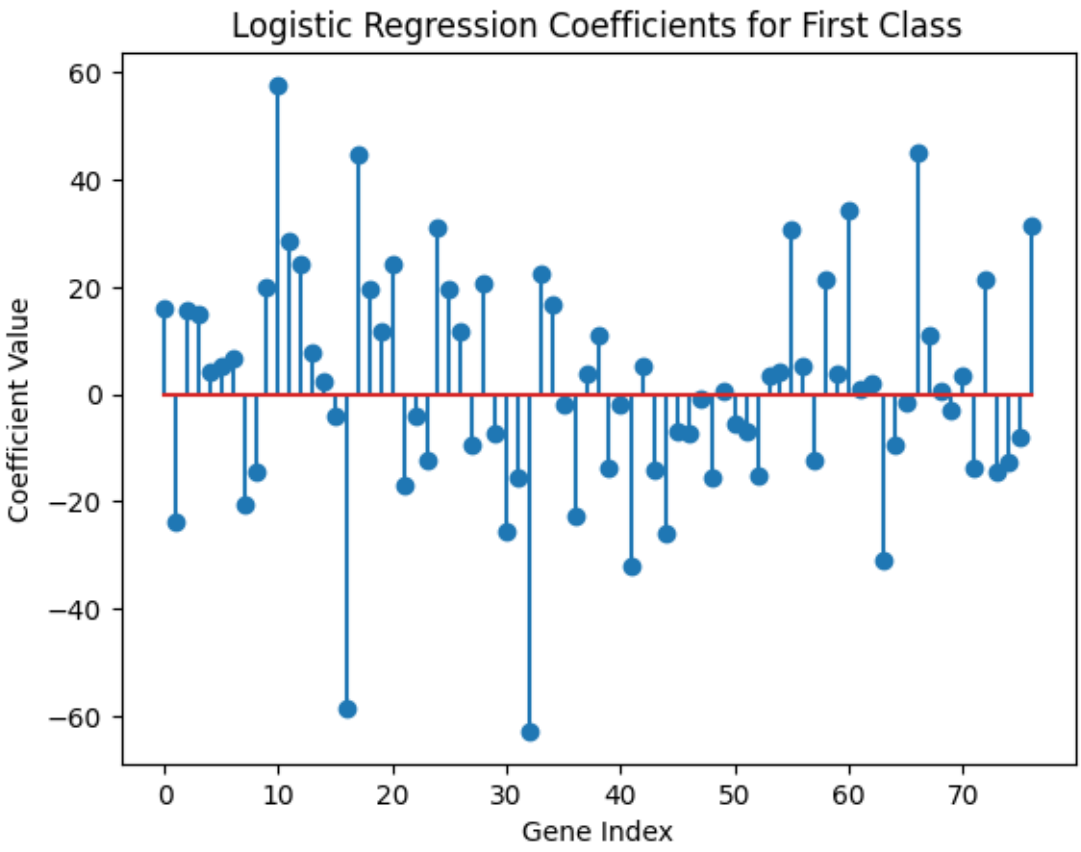
[73]:
```
from sklearn.metrics import confusion_matrix
from sklearn.model_selection import KFold

# TODO perform 10-fold cross validation using the confusion matrix
# Initialize variables
kf = KFold(n_splits=10, shuffle=True)
C_total = np.zeros((8,8)) # 8 classes
accuracies = []

# Perform k-fold cross validation
for train_idx, test_idx in kf.split(X):
    # Get train/test split for this fold
    X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
    y_train, y_test = y[train_idx], y[test_idx]

    # Convert to numpy arrays to avoid feature name warnings
    X_train_array = X_train.values
    X_test_array = X_test.values

    # Fit model and make predictions
    logreg = LogisticRegression(C=1e5, solver='lbfgs', max_iter=1000)
    logreg.fit(X_train_array, y_train)
```

```
    y_pred = logreg.predict(X_test_array)

    # Calculate confusion matrix for this fold and add to total
    C_fold = confusion_matrix(y_test, y_pred)
    C_total += C_fold

    # Calculate accuracy for this fold
    accuracies.append(np.mean(y_pred == y_test))

# Normalize confusion matrix rows to sum to 1
C = C_total.astype('float') / C_total.sum(axis=1)[:, np.newaxis]

# Print results
print("Normalized confusion matrix:")
print(np.array_str(C, precision=4, suppress_small=True))
print(f"\nMean accuracy: {np.mean(accuracies):.3f} +/- {np.std(accuracies):.
 ↪3f}")
```

```
Normalized confusion matrix:
[[0.98   0.0067 0.     0.     0.0067 0.0067 0.     0.    ]
 [0.0444 0.9037 0.     0.     0.0296 0.0222 0.     0.    ]
 [0.     0.     0.9933 0.     0.     0.     0.0067 0.    ]
 [0.     0.     0.     0.9852 0.     0.0074 0.     0.0074]
 [0.0222 0.     0.     0.     0.963  0.0074 0.     0.0074]
 [0.     0.     0.     0.     0.019  0.981  0.     0.    ]
 [0.     0.     0.0074 0.     0.     0.     0.9926 0.    ]
 [0.     0.     0.     0.     0.     0.0074 0.     0.9926]]

Mean accuracy: 0.974 +/- 0.019
```

Re-run the logistic regression on the entire training data and get the weight coefficients. This should be a 8 x 77 matrix. Create a stem plot of the first row of this matrix to see the coefficients on each of the genes.

[78]:
```
# Fit logistic regression on full dataset
logreg = LogisticRegression(C=1e5, solver='lbfgs', max_iter=1000)
logreg.fit(X, y)

# Get coefficients (8 x 77 matrix)
coef = logreg.coef_

# Create stem plot of first row (coefficients for first class)
plt.figure()
plt.stem(range(X.shape[1]), coef[0,:])
plt.xlabel('Gene Index')
plt.ylabel('Coefficient Value')
plt.title('Logistic Regression Coefficients for First Class')
plt.show()
```




### 1.7 L1-Regularization

This section is bonus.

In most genetic problems, only a limited number of the tested genes are likely influence any particular attribute. Hence, we would expect that the weight coefficients in the logistic regression model should be sparse. That is, they should be zero on any gene that plays no role in the particular attribute of interest. Genetic analysis commonly imposes sparsity by adding an l1-penalty term. Read the `sklearn` documentation on the `LogisticRegression` class to see how to set the l1-penalty and the inverse regularization strength, `C`.

Using the model selection strategies from the housing demo, use K-fold cross validation to select an appropriate inverse regularization strength.
* Use 10-fold cross validation * You should select around 20 values of `C`. It is up to you find a good range. * Make appropriate plots and print out to display your results * How does the accuracy compare to the accuracy achieved without regularization.

[15]:
```
# TODO
```

[ ]:
```

```